Applied Machine Learning Lab

B.Tech Semester 4

Experiment 07

Iris Flower Classification using Multiple Machine Learning Models

Date: 17th feb 2026

SAP ID: 590016154

Name: Vidisha

AIM

To classify iris flower species using multiple machine learning models and compare their performance based on accuracy.

THEORY

Iris classification is a supervised learning problem.
Classification Models: Logistic Regression, Decision Tree, Random Forest, SVM, KNN
Feature Scaling: Important for KNN and SVM
Confusion Matrix: Shows classification performance
Accuracy: Measures overall correctness

Commands/Code Used

```
# Import libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.svm import SVC

from sklearn.neighbors import KNeighborsClassifier


from sklearn.metrics import accuracy_score, confusion_matrix



# Load dataset

df = pd.read_csv("Iris.csv")


# Drop unnecessary column (if exists)

if "Id" in df.columns:

   df = df.drop("Id", axis=1)


# Encode target column
```

```
le = LabelEncoder()
df["Species"] = le.fit_transform(df["Species"])

# Features and target
X = df.drop("Species", axis=1)
y = df["Species"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Models
models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "SVM": SVC(),
    "KNN": KNeighborsClassifier()
}

results = {}
```

```
# Train and evaluate
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    results[name] = acc

    print(f"{name} Accuracy:", acc)

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(4,3))
    sns.heatmap(cm, annot=True, cmap="Blues", fmt="d")
    plt.title(f"Confusion Matrix - {name}")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

# Model comparison plot
results_df = pd.DataFrame(list(results.items()), columns=["Model", "Accuracy"])

plt.figure(figsize=(8,5))
sns.barplot(x="Model", y="Accuracy", data=results_df)
plt.xticks(rotation=45)
plt.title("Model Accuracy Comparison")
```

plt.show()

RESULTS

Confusion matrices generated.
Model accuracy comparison performed.

CONCLUSION

SVM achieved highest accuracy followed by other models. Model comparison helped evaluate performance.